

Clase del Módulo 4

Sitio: [Campus - Sadosky Capacitaciones](#)
Curso: Introducción al mundo de la programación paralela para HPC_V1
Libro: Clase del Módulo 4

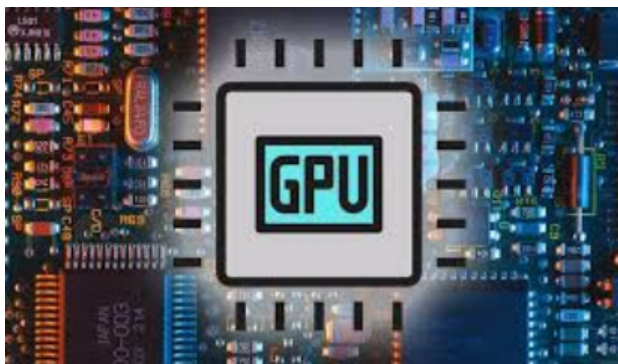
Imprimido por: Pablo Carrai
Día: jueves, 11 de junio de 2026, 15:39

Tabla de contenidos

- 1. Introducción
- 2. Diferencias conceptuales entre CPU y GPU
- 3. Modelo CUDA
 - Comandos y propiedades
 - Instrucciones en CUDA
 - Estructura general de un programa CUDA
 - Identificación de threads y bloques
 - Memoria en CUDA
 - Otros ajustes y funciones
- 4. Modelo de programación OpenCL
- 5. Problemas Clásicos
- 6. Cierre

1. Introducción

En las últimas dos décadas, las Unidades de Procesamiento Gráfico (GPU) han evolucionado desde simples aceleradores de renderizado hasta convertirse en plataformas de cómputo masivamente paralelas. Originalmente diseñadas para realizar millones de operaciones en píxeles y vértices por segundo, las GPU modernas son capaces de ejecutar miles de *threads* concurrentes sobre grandes volúmenes de datos, ofreciendo una potencia de cálculo que supera ampliamente a la de las CPU tradicionales en tareas altamente paralelizables.



La programación en GPU se ha convertido en un pilar de la Computación de Alto Desempeño (HPC), impulsando áreas como la inteligencia artificial, la simulación científica, la analítica de datos, la biología computacional y la visualización avanzada. Mediante lenguajes como CUDA (*Compute Unified Device Architecture*) y OpenCL (*Open Computing Language*), los programadores pueden aprovechar directamente el hardware de las GPU para ejecutar algoritmos de propósito general (GPGPU, *General-Purpose GPU Computing*). En este último módulo exploraremos la arquitectura de CUDA y lo compararemos con OpenCL. ¡Comencemos!

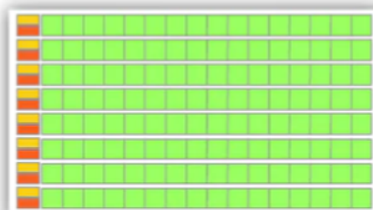
2. Diferencias conceptuales entre CPU y GPU

CPU



Prioriza la ejecución rápida de una única tarea compleja, ideal para control de flujo, lógica condicional y tareas dependientes.

GPU



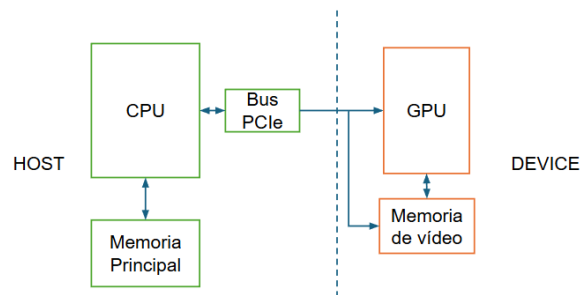
Sacrifica esa flexibilidad para lograr mayor rendimiento total al procesar grandes volúmenes de datos en paralelo, algo común en procesamiento de gráficos, simulaciones o aprendizaje automático.

La CPU es como un “cerebro generalista” capaz de manejar cualquier tipo de instrucción, mientras que la GPU es un “ejército de trabajadores especializados” que ejecutan en masa la misma operación sobre diferentes datos.

Jerarquía de memoria

La gestión de memoria también refleja el objetivo de cada arquitectura. La CPU está diseñada para minimizar la latencia de acceso a memoria, mientras que la GPU maximiza el ancho de banda, es decir, cuántos datos puede mover simultáneamente.

Aspecto	CPU	GPU
Caché	Jerarquía grande (L1, L2, L3) optimizada para baja latencia.	Caché más pequeña; la prioridad es ancho de banda.
Acceso a memoria	Acceso aleatorio rápido y coherente.	Acceso eficiente cuando los <i>threads</i> leen datos contiguos (coalescencia).
Memoria principal	Control directo del sistema operativo.	Controlada por el <i>driver</i> de GPU (memoria global, compartida, constante, local).



Como se mencionó en el módulo 1, las arquitecturas utilizadas para programar las GPU se pueden dividir en dos partes. El HOST contiene el procesador (CPU) conectado a la memoria principal y al puerto PCI *express*, que a su vez se conecta a la GPU. A la tarjeta gráfica se la llama DEVICE, la cual tiene una memoria (*device memory*) y está formada por varios multiprocesadores.



En otras palabras, en los sistemas modernos, la CPU y la GPU no compiten, sino que se complementan:

- La CPU coordina, gestiona y controla el flujo general del programa.
- La GPU ejecuta los cálculos intensivos sobre grandes volúmenes de datos.

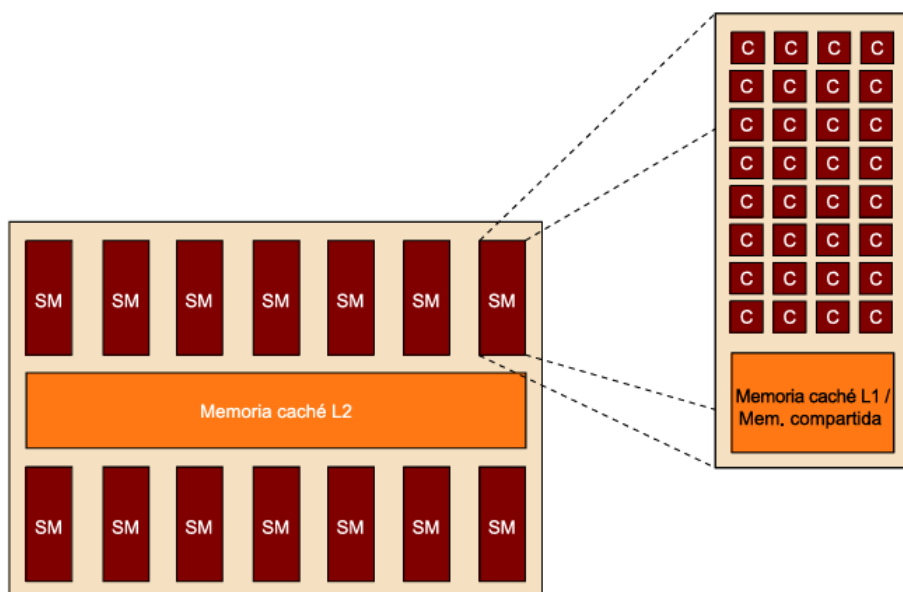
Este modelo híbrido se conoce como computación heterogénea.

En entornos de HPC, IA o simulación científica, aprovechar ambos procesadores de manera coordinada es esencial para lograr la máxima eficiencia y escalabilidad.

3. Modelo CUDA

Conceptos de arquitectura

Los modelos de la empresa Nvidia implementan la arquitectura Fermi. Esta incluye un conjunto de *Streaming Multiprocessors* (SM) que comparten memoria caché L2. Los SM operan independientemente y están compuestos por 32 núcleos (*cores*), una memoria pequeña y unidades de ejecución. Cada uno de ellos puede ejecutar una instrucción entera o en punto flotante por ciclo de reloj.



Fuente de la imagen: Guim, F. y Rodero, I., Esquema simplificado de la arquitectura Fermi, Arquitecturas basadas en computación gráfica, Universidad Oberta de Catalunya, p. 36. <https://progc-unlam.com.ar/material-clase/GPU/Arquitecturas-basadas-en-computacion-grafica.pdf>.



Un *core* es la unidad de cálculo más pequeña en una GPU. A diferencia de los *core* de la CPU, los *core* de la GPU están optimizados para operaciones de punto flotante (FLOP).

3.2 Modelo de programación CUDA

CUDA es una sigla por *Compute Unified Device Architecture*. Es una plataforma y modelo de programación desarrollada por NVIDIA que permite aprovechar la potencia de las GPU para ejecutar algoritmos de propósito general (GPGPU, *General Purpose GPU Computing*). El modelo de programación CUDA se basa en la colaboración entre la CPU (*host*) y la GPU (*device*), cada una con su propio espacio de memoria y rol específico en la ejecución.



Veamos un video de introducción a CUDA que explica la organización de *kernels* y *threads* dentro de bloques y *grids*.



Si necesitás instrucciones para configurar subtítulos en castellano hacé [clic aquí](#).

Fuente: Josh Holloway, *CUDA Teaching Center Oklahoma State University* (5 de mayo 2019). *Intro to CUDA (part 1): High Level Concepts*. [Video]. YouTube.

Ahora veamos algunos conceptos centrales y su descripción:

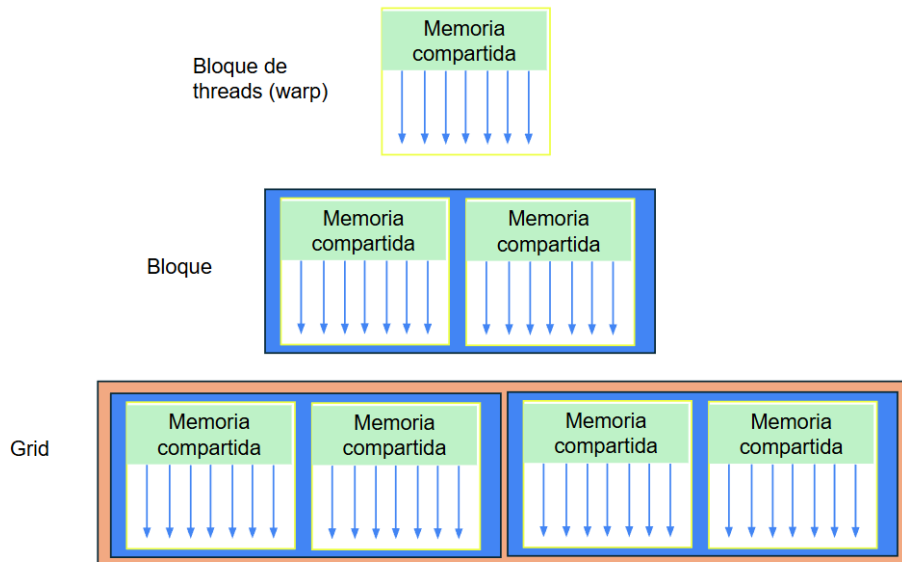
La GPU no ejecuta *threads* individualmente, sino en grupos de *warps*. Cada *thread* dentro del *warp* ejecuta la misma

instrucción (aunque con datos diferentes). Este modelo se conoce como *Single Instruction, Multiple Threads (SIMT)*.



CUDA organiza los *threads* en una jerarquía:

Grid → Bloques → Warps → Threads



Comandos y propiedades



Veamos un video de la profesora Verónica Gil Costra que nos muestra algunos comandos iniciales para obtener información de la GPU.



El comando `nvidia-smi` muestra información general del GPU, como el modelo (por ejemplo, NVIDIA RTX 3090 o Tesla T4), cantidad de memoria total y usada y la versión del *driver* y CUDA.

```
$> nvidia-smi
```

```
NVIDIA-SMI 418.87.00      Driver Version: 418.87.00      CUDA Version: 10.1
```

GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.
0	GeForce GTX 108...	off	00000000:23:00.0	off		N/A
17%	56C	P0	67W / 250W	10MiB / 11176MiB	0%	Default


```
Processes:
```

GPU	PID	Type	Process name	GPU Memory Usage
No running processes found				

Por su parte, el comando `nvcc --version` entrega información de la versión del compilador.

```
$> nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2019 NVIDIA Corporation
Built on Sun_Jul_28_19:07:16_PDT_2019
Cuda compilation tools, release 10.1, v10.1.243
```

Para compilar un código en CUDA se utiliza el siguiente comando:

```
$> nvcc programa.cu -o ejecutable
```

Mientras que para ejecutar un programa se utiliza:

```
$> ./ejecutable
```

Y finalmente, si queremos conocer cuántos *threads* por bloques y cuántos bloques por *grid*, es posible utilizar la siguiente instrucción:

```
cudaGetDeviceProperties
```

Veamos un ejemplo:

```
#include <stdio.h>
#include <cuda_runtime.h>
int main()
{
    cudaDeviceProp properties;
    cudaError_t err = cudaGetDeviceProperties(&properties, 0); // primer dispositivo
    if (err != cudaSuccess) {
        printf("Error obteniendo propiedades de la GPU:
               %s\n", cudaGetErrorString(err));
        return 1;
    }
    printf("Máximo de threads por bloque: %d\n", properties.maxThreadsPerBlock);
    printf("Máximo de bloques por cuadrícula (X, Y, Z): (%d, %d, %d)\n",
           properties.maxGridSize[0],
           properties.maxGridSize[1],
           properties.maxGridSize[2]);
    return 0;
}
```

Salida:

- Máximo de hilos por bloque: 1024
- Máximo de bloques por cuadrícula (X, Y, Z): (2147483647, 65535, 65535)

Instrucciones en CUDA

El lenguaje CUDA extiende C/C++ incorporando palabras clave e instrucciones especiales que permiten controlar cómo se ejecuta el código en la GPU. Estas extensiones definen dónde se ejecuta una función (en CPU o GPU), cómo se lanzan los *threads* y cómo se manejan los distintos tipos de memoria y sincronización.

Calificador	Se ejecuta en	Se llama desde	Descripción
<code>__global__</code>	Device (GPU)	Host (CPU)	Define una función <i>kernel</i> . No puede devolver valores.
<code>__device__</code>	Device (GPU)	Device (GPU)	Función auxiliar usada por otros <i>kernels</i> .
<code>__host__</code>	Host (CPU)	Host (CPU)	Indica que la función se ejecuta en CPU (por defecto).

__host__ __device__	CPU y GPU	CPU o GPU	Permite compilar la misma función para ambos entornos.
---------------------	-----------	-----------	--



Veamos un ejemplo de cómo multiplicar dos matrices $A*B$ y dejar el resultado en un arreglo C .

Fundación Sadosky



Mirar en


```

#include <stdio.h>
#include <cuda_runtime.h>
// =====
// Función de dispositivo: se ejecuta en GPU,
// pero se llama desde otro kernel (no desde CPU)
// =====
__device__ float multiplicar(float a, float b) {
    return a * b;
}

// =====
// Función kernel: se ejecuta en la GPU pero se invoca desde la CPU
// =====
__global__ void vectorM(float *A, float *B, float *C, int N) {
    // Cálculo del índice global del thread
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // Verificación de límites (para no acceder fuera del vector)
    if (i < N)
        C[i] = multiplicar(A[i], B[i]);
}

// =====
// Función de host: se ejecuta en CPU (por defecto)
// =====
__host__ void mostrarMensaje() {
    printf("Ejecutando en CPU\n");
}

// =====
// Función principal: se ejecuta en CPU
// =====
int main() {
    mostrarMensaje(); // Llamada a función de host
    // Número de elementos en los vectores
    int N = 1024;
    size_t size = N * sizeof(float);
    // Reserva de memoria en el host (CPU)
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);
    float *h_C = (float*)malloc(size);
    // Inicialización de datos
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }
    // Reserva de memoria en el device (GPU)
    float *d_A, *d_B, *d_C;
    cudaMalloc((void*)&d_A, size);
    cudaMalloc((void*)&d_B, size);
    cudaMalloc((void*)&d_C, size);
    // Copiar datos del host a la GPU
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    // Configuración de la ejecución en la GPU
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    // Llamada al kernel (ejecutado en GPU)
    vectorM<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, N);
    // Sincronizar la GPU antes de continuar
    cudaDeviceSynchronize();

    // Copiar resultados desde GPU a CPU
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    // Mostrar resultados
    printf("Resultado de ejemplo:\n");
    printf("C[0] = %.1f\n", h_C[0]);
    printf("C[%d] = %.1f\n", N-1, h_C[N-1]);
    // Liberar memoria
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
}

```

```
    free(h_A);  
    free(h_B);  
    free(h_C);  
    return 0;  
}
```

Salida:

- Ejecutando en CPU
- Resultado de ejemplo:
- `C[0] = 0.0`
- `C[1023] = 2093058.0`



La instrucción `cudaDeviceSynchronize()` fuerza a la CPU a esperar hasta que la GPU termine todas sus tareas pendientes.

Finalmente, CUDA provee funciones específicas para reservar, copiar y liberar memoria en el *device*. Las vemos en el siguiente cuadro:

Instrucción	Descripción
<code>cudaMalloc()</code>	Reserva memoria en la GPU
<code>cudaMemcpy()</code>	Copia datos entre host y device
<code>cudaFree()</code>	Libera memoria en la GPU
<code>cudaMemset()</code>	Inicializa bloques de memoria

Estructura general de un programa CUDA

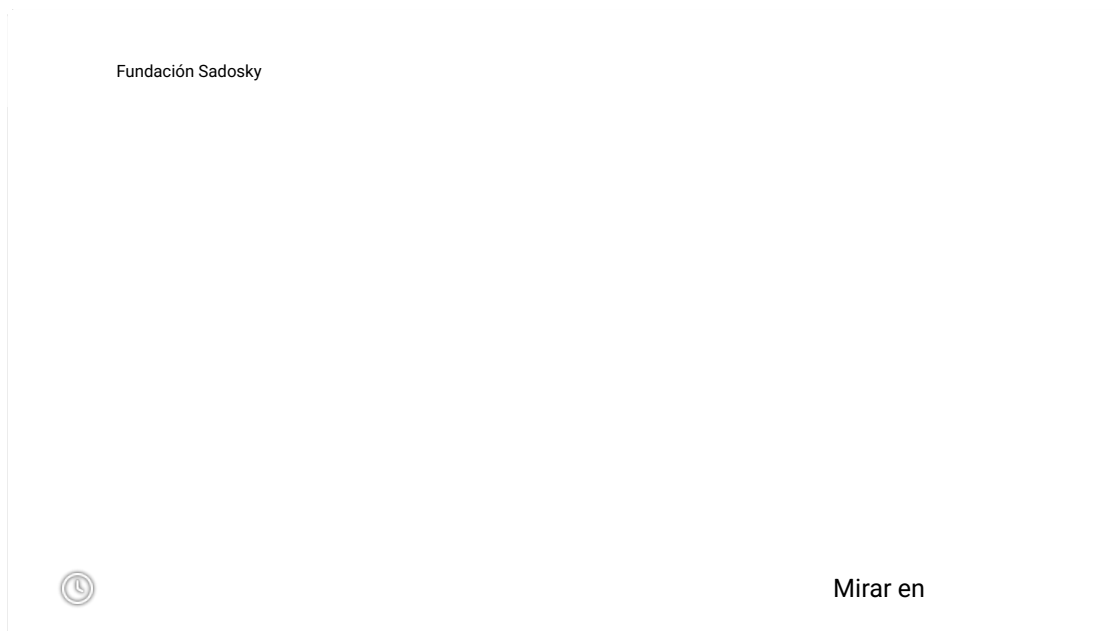
Un programa CUDA tiene dos partes bien diferenciadas:

- > **Código ejecutado en el host (CPU)**
- > **Código ejecutado en el device (GPU)**

Como ya habíamos visto en el módulo 1, un *kernel* es una función que se ejecuta en paralelo por muchos *threads* dentro de la GPU.



Veamos a continuación un ejemplo de cómo realizar la suma de dos vectores, y dejar un resultado en un vector de salida utilizando una GPU:



Veamos el código:

```

t// vector_add.cu
#include <stdio.h>
#include <cuda_runtime.h>

//Kernel CUDA: ejecutado por la GPU
__global__ void vectorAdd(float *A, float *B, float *C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N)
        C[i] = A[i] + B[i];
}

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);
    // Reservar memoria en el host
    float *h_A = (float*) malloc(size);
    float *h_B = (float*) malloc(size);
    float *h_C = (float*) malloc(size);
    // Inicializar datos
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }
    // Reservar memoria en el device (GPU)
    float *d_A, *d_B, *d_C;
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // Copiar datos del host al device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Definir la configuración de ejecución
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    // Lanzar el kernel
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, N);

    // Copiar resultados de vuelta al host
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Mostrar algunos resultados
    printf("C[0] = %.2f\n", h_C[0]);
    printf("C[%d] = %.2f\n", N - 1, h_C[N - 1]);

    // Liberar memoria
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);
    return 0;
}

```

La salida será:

- c[0] = 0.00
- c[1023] = 3069.00

En el código anterior se utilizan las siguientes instrucciones:

CudaMalloc: reserva de memoria en la GPU (equivalente a malloc() de la CPU).

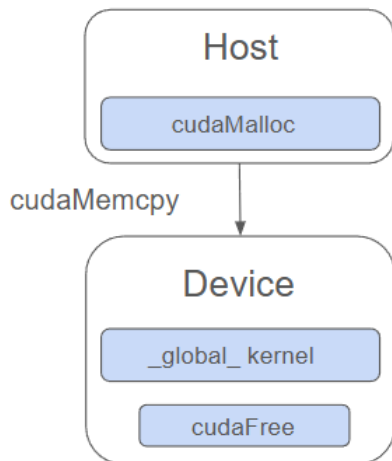
CudaMemcpy: copia de datos entre CPU y GPU.

`__global__` : se utiliza para definir un *kernel*. Le indica al compilador que esta función se ejecutará en la GPU y será llamada desde la CPU. Un *kernel* no devuelve valores (debe ser *void*). Los parámetros se copian por valor desde el host. Cada *thread* ejecuta una instancia del *kernel*.

`vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N)`: ejecuta el *kernel*. Los símbolos `<<< >>>` son la notación de ejecución de CUDA; *blocksPerGrid* indica el número de bloques por grid o malla; y *threadsPerBlock* indica el número de *threads* por bloque.

`cudaFree`: liberación de memoria en la GPU.

Veamos en el siguiente esquema las instrucciones básicas en CUDA:



Aunque no se incluya `#include <cuda_runtime.h>`, el programa puede compilar y ejecutarse correctamente porque `cuda_runtime.h` se incluye indirectamente al compilar con `nvcc`.

Para verificar errores, se utiliza el siguiente código luego de llamar a un *kernel*:

```

cudaError_t err = cudaGetLastError();
if (err != cudaSuccess)
    printf("Error CUDA: %s\n", cudaGetErrorString(err));
  
```

Identificación de threads y bloques

CUDA permite que tanto los bloques como los threads dentro de un bloque tengan hasta tres dimensiones: x, y, z. Esto ayuda a mapear datos multidimensionales (imágenes, volúmenes, etc.) a la GPU de manera natural.



Los *threads* pueden organizarse en 1D, 2D o 3D:

- 1D: `threadIdx.x` acceso lineal.
- 2D: `threadIdx.x`, `threadIdx.y` acceso a matrices.
- 3D: `threadIdx.x`, `threadIdx.y` `threadIdx.z` acceso a volúmenes.

De forma similar, los bloques se organizan en la *grid* o malla.

- 1D: `blockIdx.x`
- 2D: `blockIdx.x`, `blockIdx.y`
- 3D: `blockIdx.x`, `blockIdx.y` `blockIdx.z`

Además, dentro del *kernel*, CUDA provee variables especiales para identificar cada *thread*:

`blockDim.x`: cantidad de *threads* por bloque en la dimensión x.

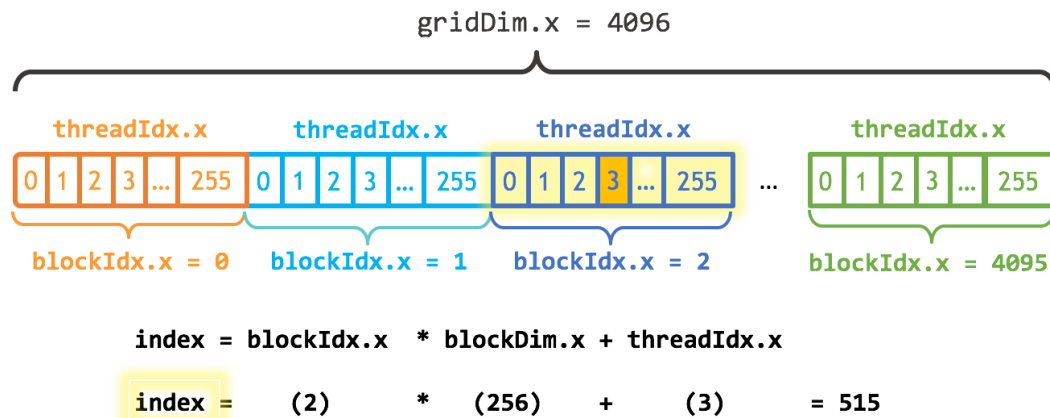
`gridDim.x`: cantidad total de bloques en la dimensión x.

Para acceder a un dato en un arreglo multidimensional, se puede calcular el índice global:

Para 1D lineal	<code>int id = blockIdx.x * blockDim.x + threadIdx.x;</code>
Para 2D	<code>int x = blockIdx.x * blockDim.x + threadIdx.x;</code> <code>int y = blockIdx.y * blockDim.y + threadIdx.y;</code>
Para 3D	<code>int x = blockIdx.x * blockDim.x + threadIdx.x;</code> <code>int y = blockIdx.y * blockDim.y + threadIdx.y;</code> <code>int z = blockIdx.z * blockDim.z + threadIdx.z;</code>

Esto es lo que permite que cada *thread* acceda a su parte única de datos. Generalmente, "x" referencia a la dimensión horizontal o más rápida, "y" referencia la segunda dimensión (vertical en imágenes o matrices) y "z" referencia la tercera dimensión (profundidad, volúmenes).

En la siguiente figura, se ve gráficamente cómo se puede calcular el índice del *thread* 3 del bloque 2. En este ejemplo, cada bloque tiene 256 *threads*, por lo tanto, `blockDim.x=256`.



Fuente de imagen: (s.f.). *GPU concepts*. <https://enccs.github.io/gpu-programming/4-gpu-concepts/>.



Para cerrar esta sección, veamos a continuación un video que explica de manera gráfica cómo identificar *threads* en la arquitectura de la GPU:



Si necesitás instrucciones para configurar subtítulos en castellano hacé [clic aquí](#).

Fuente: Fuente: Josh Holloway, *CUDA Teaching Center Oklahoma State University* (5 de mayo 2019). *Intro to CUDA (part 4): Indexing Threads within Grids and Blocks* [Video]. YouTube.

Memoria en CUDA

El modelo de memoria en CUDA es jerárquico y está diseñado para equilibrar velocidad y capacidad:

Tipo de memoria	Acceso	Velocidad	Alcance
Global	Todos los <i>threads</i>	Lenta	Toda la GPU
Compartida (<i>shared</i>)	Solo <i>threads</i> del mismo bloque	Muy rápida	Bloque
Local / Registros	Solo el <i>thread</i> actual	Muy rápida	<i>Thread</i>
Constante / De solo lectura	Todos los <i>threads</i>	Rápida (cacheada)	Global

Los *threads* dentro de un mismo bloque pueden cooperar mediante memoria compartida y sincronizarse usando:

```
__syncthreads();
```

Esto garantiza que todos los *threads* del bloque alcancen el mismo punto antes de continuar.

Como se explicó anteriormente, CUDA permite declarar diferentes tipos de memoria dentro del *kernel* o a nivel global.

Calificador	Tipo de memoria	Alcance	Velocidad	Ejemplo
<code>__shared__</code>	Memoria compartida	Bloque	Muy rápida	<code>__shared__ float temp[256];</code>
<code>__device__</code>	Memoria global	GPU	Media	<code>__device__ float datos[1024];</code>
<code>__constant__</code>	Memoria constante	Toda la GPU	Cacheada	<code>__constant__ float PI = 3.1415;</code>



Veamos el video en que la profesora Verónica Gil Costa nos muestra el uso de la memoria compartida a través de la directiva *shared* de CUDA:

Fundación Sadosky



Mirar en


```

#include <stdio.h>
#include <cuda_runtime.h>
// =====
// Kernel CUDA: usa memoria compartida y sincronización
// =====
__global__ void sumaParcial(float *A, float *B, float *C, int N)
{
    __shared__ float temp[256]; // Memoria compartida para los threads del bloque
    int i = threadIdx.x;        // Índice local dentro del bloque
    // Cada thread suma los elementos correspondientes
    if (i < N)
        temp[i] = A[i] + B[i];
    /// Esperar a que todos los threads terminen de escribir en temp[]
    __syncthreads();
    // Cada thread copia su resultado parcial al vector global de salida
    if (i < N)
        C[i] = temp[i];
}
// =====
// Función principal ejecutada en CPU (Host)
// =====
int main() {
    int N = 256; // Tamaño de los vectores
    size_t size = N * sizeof(float);
    // Reservar memoria en CPU (Host)
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);
    float *h_C = (float*)malloc(size);
    // Inicializar datos
    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }
    // Reservar memoria en GPU (Device)
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    // Copiar datos del Host a Device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Lanzar kernel: 1 bloque con 256 threads
    sumaParcial<<<1, 256>>>(d_A, d_B, d_C, N);

    // Sincronizar ejecución
    cudaDeviceSynchronize();

    // Copiar resultados de Device a Host
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    // Mostrar resultados de ejemplo
    printf("Ejemplo de suma parcial usando memoria compartida:\n");
    printf("C[0] = %.1f\n", h_C[0]);
    printf("C[%d] = %.1f\n", N-1, h_C[N-1]);
    // Liberar memoria
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);
    return 0;
}

```

La salida será:

- Ejemplo de suma parcial usando memoria compartida:
- `c[0] = 0.0`

- `c[255] = 765.0`

En este ejemplo, la memoria `__shared__` es un espacio de memoria rápida (similar a una *caché*) compartida entre todos los *threads* del mismo bloque. Cada bloque tiene su propia copia privada de esta memoria. Todos los *threads* pueden leer y escribir en ella, pero se debe sincronizar para evitar conflictos.

Aquí, los 256 *threads* del bloque escriben en `temp[i]`, esperan (`__syncthreads()`) y finalmente, cada *thread* copia su valor a `C[i]`.

Si no se utiliza `__shared__`, el código funciona igual, pero accede directamente a memoria global, que es más lenta y, por lo tanto, no aprovecha la caché rápida compartida.

Un ejemplo sin `__shared__` sería:

```
__global__ void sumaDirecta(float *A, float *B, float *C) {
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}
```

Otros ajustes y funciones

Selección del tamaño de bloque

La elección del tamaño de bloque (`blockDim`) es uno de los ajustes más críticos para el rendimiento. Se recomienda usar múltiplos de 32 (tamaño de *warp*) o probar tamaños comunes: 128, 256 o 512 *threads* por bloque. Además, es recomendable evitar bloques demasiado grandes que excedan la memoria compartida disponible.

Ejemplo:

```
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, N);
```

Una elección adecuada de `threadsPerBlock` puede mejorar significativamente el *throughput* del *kernel*.



Throughput del *kernel* es la tasa o cantidad de trabajo completado en una unidad de tiempo específica.

Coalescencia de accesos a memoria

La memoria global de la GPU es muy rápida en ancho de banda, pero tiene alta latencia. Para aprovechar su velocidad, los accesos deben ser coalescentes, es decir, los *threads* consecutivos deben acceder a posiciones contiguas en memoria.

Mal acceso (no coalescente):

```
C[i] = A[2*i]; // saltos de memoria
```

Buen acceso (coalescente):

`C[i] = A[i]; // accesos contiguos`



Regla práctica:

Cada *warp* (32 *threads*) debe leer/escribir datos consecutivos para maximizar el ancho de banda.

Evitar ramificaciones divergentes

Dentro de un *warp*, todos los *threads* ejecutan la misma instrucción en cada ciclo. Si hay condicionales (if/else) donde los *threads* toman caminos distintos, ocurre divergencia de flujo y el rendimiento disminuye.

Un ejemplo de código con divergencia:

```
if (threadIdx.x % 2 == 0)
    A[i] *= 2;
else
    A[i] += 1;
```

Un ejemplo de alternativa sin divergencia:

```
A[i] = (A[i] * 2) * (threadIdx.x % 2 == 0) + (A[i] + 1) * (threadIdx.x % 2 != 0);
```

Funciones atómicas en CUDA

Las funciones atómicas son una herramienta clave que CUDA ofrece al programador y que permiten evitar problemas de carrera tal como se definió en la sección 3.2 del módulo 1. Garantizan que una operación de "lectura-modificación-escritura" sobre una variable en memoria global (o compartida) ocurra de forma indivisible.

Si dos *threads* intentan ejecutar una función atómica sobre la misma dirección de memoria al mismo tiempo, la GPU garantiza la integridad de los datos ordenando las operaciones, lo que significa que una operación se completará antes que la otra, sin causar problemas de carrera.

Función	Qué hace
<code>atomicAdd(address, val)</code>	<code>*address = *address + val;</code>
<code>atomicSub(address, val)</code>	<code>*address = *address - val;</code>
<code>atomicAnd(address, val)</code>	<code>*address = *address & val;</code> (AND a nivel de bits)
<code>atomicOr(address, val)</code>	<code>*address = *address val;</code> (OR a nivel de bits)
<code>atomicXor(address, val)</code>	<code>*address = *address ^ val;</code> (XOR a nivel de bits)
<code>atomicMin(address, val)</code>	<code>*address = min(*address, val);</code>
<code>atomicMax(address, val)</code>	<code>*address = max(*address, val);</code>
<code>atomicExch(address, val)</code>	<code>*address = val;</code>
<code>atomicInc(address, val)</code>	<code>*address = (*address >= val) ? 0 : (*address + 1);</code>

```
atomicDec(address, val)
```

```
*address = ((*address == 0) || (*address > val)) ? val : (*address - 1);
```

4. Modelo de programación OpenCL

OpenCL sigue un esquema muy parecido al de CUDA, aunque con terminología diferente.

Concepto CUDA	Concepto OpenCL	Descripción
Host	Host	CPU que controla la ejecución
Device	Device	GPU o acelerador que ejecuta el <i>kernel</i>
Kernel	Kernel	Función paralela ejecutada en la GPU
Grid	NDRange	Rango de trabajo total (todos los <i>threads</i>)
Block	Work-group	Conjunto de <i>threads</i> que cooperan
Thread	Work-item	<i>Thread</i> individual de ejecución

Un kernel OpenCL es una función que se ejecuta en todos los work-items en paralelo, muy similar al modelo de CUDA.

Arquitectura general en OpenCL

Un programa OpenCL se divide en dos partes:

1. Código Host (CPU): escrito en C o C++. Gestiona la plataforma, dispositivos, memoria y lanzamiento del kernel.
2. Código Device (GPU): escrito en un lenguaje tipo C, compilado y ejecutado por la GPU. Se define con la palabra clave `__kernel`.

Veamos un ejemplo:

```

//Código del kernel (GPU)
// archivo: vector_add.cl
__kernel void vector_add(__global const float *A, __global const float *B,
                        __global float *C, const int N)
{
    int i = get_global_id(0);
    if (i < N)
        C[i] = A[i] + B[i];
}

//Código Host (CPU)
#include <CL/cl.h>
#include <stdio.h>
#include <stdlib.h>
#define N 1024
int main() {
    float A[N], B[N], C[N];
    for (int i = 0; i < N; i++) {
        A[i] = i * 1.0f;
        B[i] = i * 2.0f;
    }
    // Obtener plataforma y dispositivo
    cl_platform_id platform;
    cl_device_id device;
    clGetPlatformIDs(1, &platform, NULL);
    clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL);
    // Crear contexto y cola
    cl_context context = clCreateContext(NULL, 1, &device, NULL, NULL, NULL);
    cl_command_queue queue = clCreateCommandQueue(context, device, 0,
    NULL);

    //Crear buffers
    cl_mem bufA = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(A),
                                NULL, NULL);
    cl_mem bufB = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(B),
                                NULL, NULL);
    cl_mem bufC = clCreateBuffer(context, CL_MEM_WRITE_ONLY, sizeof(C),
                                NULL, NULL);

    // Copiar datos al device
    clEnqueueWriteBuffer(queue, bufA, CL_TRUE, 0, sizeof(A), A, 0, NULL,
                        NULL);
    clEnqueueWriteBuffer(queue, bufB, CL_TRUE, 0, sizeof(B), B, 0, NULL,
                        NULL);

    // Cargar y compilar el kernel
    FILE *f = fopen("vector_add.cl", "r");
    fseek(f, 0, SEEK_END);
    size_t size = ftell(f);
    rewind(f);
    char *source = (char*)malloc(size);
    fread(source, 1, size, f);
    fclose(f);
    cl_program program = clCreateProgramWithSource(context, 1, (const
                                                char**)&source, &size, NULL);
    clBuildProgram(program, 1, &device, NULL, NULL, NULL);
    cl_kernel kernel = clCreateKernel(program, "vector_add", NULL);
    //Asignar argumentos y ejecutar
    clSetKernelArg(kernel, 0, sizeof(cl_mem), &bufA);
    clSetKernelArg(kernel, 1, sizeof(cl_mem), &bufB);
    clSetKernelArg(kernel, 2, sizeof(cl_mem), &bufC);
    clSetKernelArg(kernel, 3, sizeof(int), &N);
    size_t globalSize = N;
    clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &globalSize, NULL, 0,
                        NULL, NULL);

    // Leer resultados
    clEnqueueReadBuffer(queue, bufC, CL_TRUE, 0, sizeof(C), C, 0, NULL,
                        NULL);

    printf("C[0] = %.2f, C[%d] = %.2f\n", C[0], N-1, C[N-1]);
    //Liberar recursos
    clReleaseMemObject(bufA);

```

```
    clReleaseMemObject(bufB);  
    clReleaseMemObject(bufC);  
    clReleaseProgram(program);  
    clReleaseKernel(kernel);  
    clReleaseCommandQueue(queue);  
    clReleaseContext(context);  
    free(source);  
    return 0;  
}
```

5. Problemas Clásicos

Memoria CPU vs GPU

El primer problema clásico al programar con CUDA en una GPU es confundir la memoria del *host* con la del *device*.



Veamos este problema típico al trabajar con CUDA:

Fundación Sadosky



Mirar en

Ahora revisemos el código:

```

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void count_race(int *data, int *counter, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N && data[idx] > 0) {
        // PROBLEMAS DE CARRERA:
        // varios hilos leen counter[0] simultáneamente
        counter[0] = counter[0] + 1;
    }
}

int main() {
    int N = 1000;
    int h_data[N];
    // Inicializamos datos
    for (int i = 0; i < N; i++) h_data[i] = 1; // todos positivos
    int *d_data, *d_counter;
    int h_counter = 0;
    cudaMalloc(&d_data, N*sizeof(int));
    cudaMalloc(&d_counter, sizeof(int));

    cudaMemcpy(d_data, h_data, N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_counter, &h_counter, sizeof(int), cudaMemcpyHostToDevice);
    count_race<<<10, 128>>>(d_data, d_counter, N);
    cudaDeviceSynchronize();
    cudaMemcpy(&h_counter, d_counter, sizeof(int), cudaMemcpyDeviceToHost);
    printf("Resultado incorrecto: %d (debería ser %d)\n", h_counter, N);
    cudaFree(d_data);
    cudaFree(d_counter);
    return 0;
}

```

El *kernel* se ejecuta en la GPU. Cuando se lanza `add_one`, se pasa `h_data`, que es una dirección de memoria en la RAM de la CPU. La GPU no puede acceder directamente a esa dirección. El *kernel* estará leyendo y escribiendo en una ubicación de memoria basura (o inválida) dentro de la memoria de la GPU que casualmente tiene el mismo valor numérico que la dirección de la CPU.

Veamos entonces la solución:

```

__global__ void count_atomic(int *data, int *counter, int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N && data[idx] > 0) {
        // OPERACIÓN ATÓMICA: evita la problemas de carrera
        atomicAdd(counter, 1);
    }
}

```

Problema de carrera

Como se mencionó en el capítulo 3 del Módulo 1, otro problema típico son los problemas de carrera. Este tipo de error da un resultado diferente cada vez que se ejecuta el código. Supongamos que se quiere contar cuántos números son mayores a 10 en un arreglo y veamos cómo se genera una condición de carrera y una solución a ello.



En el siguiente video veremos este ejemplo en que se genera una condición de carrera, y cómo se puede solucionar.



Veamos el código:

```
__global__ void count_greater(int* data, int* counter)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (data[idx] > 10) { // ¡PROBLEMA DE CARRERA!
        // Varios threads intentan hacer esto AL MISMO TIEMPO
        counter[0] = counter[0] + 1;
    }
}
```

La operación `counter[0] = counter[0] + 1` no es atómica. En realidad son tres pasos:

1. Leer: `counter[0]` (por ejemplo: lee el valor 5).
2. Modificar: por ejemplo, calcular $5 + 1 = 6$.
3. Escribir: escribir 6 en `counter[0]`.

Veamos entonces la solución:

```
__global__ void count_greater(int* data, int* counter)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (data[idx] > 10) { // Solucionado
        atomicAdd(counter, 1);
    }
}
```

Sincronización

La CPU no espera a la GPU. Este problema se produce cuando se ejecuta un *kernel* y la CPU inmediatamente intenta leer el resultado.



En el siguiente video veremos este problema que se produce si no incluimos una instrucción de sincronización luego de llamar al *kernel* de la GPU.

Fundación Sadosky



Mirar en

Veamos el código:

```
__global__ void slow_kernel(float* data) {  
    // Simular un trabajo muy largo  
    for(long i = 0; i < 100000000; i++); data[0] = 123.0f;  
}  
  
int main() {  
    float* data;  
    // Memoria Unificada, accesible desde CPU y GPU  
    cudaMallocManaged(&data, sizeof(float));  
    data[0] = 0.0f;  
    // ejecuta el kernel (esto es ASÍNCRONO)  
    slow_kernel<<<1, 1>>>(data);  
    //CUIDADO! La CPU sigue ejecutando  
    // El kernel AÚN NO HA TERMINADO, pero la CPU lee el valor  
    printf("Resultado de la GPU: %f\n", data[0]); // Imprime 0.0f  
    cudaFree(data);  
    return 0;  
}
```

La solución a este problema es incluir la siguiente instrucción luego de la llamada al *kernel*:

```
cudaDeviceSynchronize();
```

6. Cierre



¡Felicitaciones, llegaste hasta el final del último módulo del curso!

En este módulo hemos presentado los fundamentos de la programación en GPU como un componente central de la computación de alto desempeño moderna. A partir de la comparación conceptual entre CPU y GPU, analizamos cómo las diferencias arquitectónicas influyen directamente en los modelos de programación, el manejo de memoria y las estrategias de paralelización. Esta comprensión es esencial para que podamos identificar qué tipos de problemas se benefician del paralelismo masivo ofrecido por las GPU.

También estudiamos en detalle el modelo de programación CUDA, abordando la organización jerárquica de *threads*, bloques y grillas, el modelo de memoria, la sincronización y el uso de operaciones atómicas. A través de ejemplos prácticos, observamos patrones comunes de programación, así como errores frecuentes como problemas de carrera, accesos incorrectos a memoria y problemas de sincronización, junto con sus soluciones. Estos aspectos resultan críticos para desarrollar aplicaciones correctas y eficientes sobre GPU.

Finalmente, vimos que OpenCL es una alternativa multiplataforma que permite extender el cómputo paralelo a diferentes tipos de dispositivos y fabricantes, reforzando la idea de computación heterogénea.



Ahora casi está en condiciones de realizar la evaluación del módulo. Pero antes, te recomendamos practicar lo aprendido en este módulo con la ejercitación. Dirígete allí haciendo [clic aquí](#).